

Supplementary Appendix: The Cost of Adaptivity: Online Fanout Selection in Cache-Conscious B-Trees

Anonymous Author(s)

1 Scope

This appendix supplies proof detail and experimental parameters for the main paper. It does not introduce new claims or experiments, and the main paper is self-contained. Reviewers need not read this appendix to evaluate the paper’s contributions.

2 Break-Even Derivation

Consider a stable phase containing T operations. Let c_0 be the average operation cost under the incumbent fanout and c_1 the cost under a candidate fanout, excluding online adaptation overhead. Define the oracle benefit per operation as

$$\Delta = c_0 - c_1.$$

Let m , ρ , and d denote the per-operation costs of monitoring, records metadata, and policy evaluation. Let R be the one-time cost of rebuilding and publishing the segment. Retaining the incumbent costs Tc_0 ; adapting costs

$$Tc_1 + T(m + \rho + d) + R.$$

Therefore adaptation is profitable exactly when

$$T(\Delta - m - \rho - d) > R.$$

If $\Delta > m + \rho + d$, the minimum profitable phase length is

$$T_{\min} = \frac{R}{\Delta - m - \rho - d}.$$

If $\Delta \leq m + \rho + d$, no finite phase length amortizes the recurring online cost. This latter case explains the measured in-memory workloads in the main paper: increasing phase length can amortize rebuilding, but cannot remove recurring monitoring, records, and policy work.

3 Potential-Function Detail

For each segment, let $\Phi_t \geq 0$ be adaptation credit immediately before operation t . Each operation contributes at most a constant γ . A rebuild of actual cost R_t is permitted only when $\Phi_t \geq R_t$, and spends R_t credit.

THEOREM 1. *The rebuild rule gives $O(1)$ amortized restructuring cost per operation.*

PROOF. Let $a_t \in \{0, R_t\}$ be actual restructuring cost and define

$$\widehat{a}_t = a_t + \Phi_{t+1} - \Phi_t.$$

Without a rebuild, $a_t = 0$ and $\Phi_{t+1} - \Phi_t \leq \gamma$, so $\widehat{a}_t \leq \gamma$. With a rebuild, $a_t = R_t$ and $\Phi_{t+1} - \Phi_t \leq \gamma - R_t$, so again $\widehat{a}_t \leq \gamma$. Summing over T operations telescopes the potential terms:

SIGMOD '27, Huntington Beach, CA, USA
2027.

Algorithm 1 Phase-aware fanout oracle

Require: Phases P , candidate fanouts $\mathcal{F}$, default fanout f_0

- 1: **for** each candidate hot fanout $f \in \mathcal{F}$ **do**
- 2: initialize all segments with fanout f_0
- 3: **for** each phase $p \in P$ **do**
- 4: obtain the known hot segment s_p
- 5: time rebuilding s_p with fanout f
- 6: replay the fixed operation sequence for p
- 7: record latency, throughput, and rebuild time
- 8: **end for**
- 9: **end for**
- 10: report the best measured candidate per phase and in aggregate

$$\sum_{t=1}^T a_t \leq T\gamma + \Phi_1 - \Phi_{T+1} \leq T\gamma + \Phi_1.$$

Thus total restructuring cost is $O(T)$, or $O(1)$ amortized per operation. $\square$

This result bounds restructuring only. It does not eliminate the recurring online term $T(m + \rho + d)$, which is why amortized rebuilding alone does not guarantee an end-to-end speedup.

4 Oracle Construction

The phase oracle is intentionally stronger than an implementable online policy: it knows the phase boundary and hot segment. For each candidate hot fanout, it rebuilds only the designated segment and reports both operation-only throughput and throughput including the measured reconfiguration time.

The oracle is an upper-bound diagnostic rather than a deployable competitor. If it cannot outperform a static baseline, online adaptation has no fanout headroom to capture. If it wins only slightly, the oracle gap bounds the total monitoring, decision, and rebuilding budget available to an online policy.

5 Experimental Parameters

Table 1 records the principal settings used by the paper’s reported experiments. Workload operation sequences are generated once per configuration and replayed across index variants so that comparisons use the same keys and operation types.

Repeated-trial summaries report the arithmetic mean, sample standard deviation, coefficient of variation, and a two-sided 95% confidence interval using Student’s t distribution with four degrees of freedom.

6 Reader Safety Boundary

The concurrent scalability experiment preloads the tree and then executes lookups and in-place updates. Existing-value updates do

Table 1: Principal benchmark parameters.

Parameter	Setting
Language standard	C++11
Compiler	GCC 6.3.0 / MinGW
Default fanout	64
Candidate fanouts	8, 16, 32, 64, 128, 256
Segments	8 equal key ranges
YCSB records	100,000
YCSB operations	100,000 per workload
Zipf parameter	0.99
Adaptation interval	5,000 operations
Repeated trials	5
Shifting phase length	30,000 operations
Thread benchmark keys	1,000,000
Thread benchmark operations	10,000,000
Thread counts	1, 4, 8, 16, 32
Thread operation mix	95% reads, 5% updates
Sliding-hotspot windows	$5 \times 1,000,000$ operations
Hot range per window	20% of key space

not delete nodes. Structural splits publish new nodes while retaining old nodes until all worker threads join, so an optimistic reader can safely validate and retry without accessing reclaimed memory. Concurrent segment replacement is outside that experiment: the adaptive rebuild measurements are single-threaded. A fully concurrent adaptive implementation would require epoch-based reclamation (or an equivalent safe-memory-reclamation mechanism), adding another online cost to the break-even model.

Generative AI Disclosure

Large language models were used to assist with code scaffolding and boilerplate, and to improve the clarity, organization, and flow of the manuscript and appendix. The authors determined the research questions and system design, conducted and validated the experiments, verified the mathematical analysis and citations, interpreted the results, and take responsibility for the work.